Programs for a Frozen Machine: The Description Length of Language and Task in Optimized Prefixes

Georgii Sirotenko
Heidelberg University
georgii.sirotenko@stud.uni-heidelberg.de

Abstract

A frozen language model is a fixed machine; a soft prefix that makes it perform a task is a program for that machine, and the shortest such program is a model-relative description length of the task. We measure that length for a frozen Qwen2.5-1.5B base model over a grid of three tasks (translation from English, factual recall, sentence continuation) and three output languages (Russian, German, English), in two units: prefix slots and variational bits under a bits-back code. Four findings. (i) Slots are not a description length: translation saturates at one slot, one trained slot beats the natural-language prompt everywhere, and a conditioning string is worth less than a slot. (ii) In bits, a translation program costs about 500 bits at 10% excess loss, a Russian recall program five times that, and English recall most of all; continuation has nothing to compress. (iii) Task, not language, is the shared factor: one centre per task with language deviations codes all eight cells for 37% fewer bits than one centre per language with task deviations, recall programs are language-specific, do not compose zero-shot and are read deep in the network, whereas translation programs are read almost entirely in the first block; inside a hierarchical prefix the language deviation is read again at blocks 16–17, which the task centre is not. (iv) The variational objective selects compact programs but the standard optimiser does not find them: a conservative step size and checkpoint selection on the full objective lower the measured frontier by 20–71% with fewer runs, which turns the open problem of Shaw et al. [14] into a recipe at this scale. All numbers regenerate from the released tables.

1 Introduction

Prompt tuning [9] and prefix tuning [11] adapt a frozen language model by optimising a few continuous vectors prepended to the input. Read as engineering, these vectors are parameters. Read through the lens of algorithmic information theory they are programs: the frozen model is the interpreter, the prefix is the program, the input is the data and the target is the result. Shaw et al. [14] make this reading precise, showing that prompt tokens form a program tape for a Transformer interpreter and that the variational code length of a prompt is an asymptotically optimal description-length objective. Cosma [3] define prompting complexity as a model-relative, resource-bounded Kolmogorov complexity. Both papers measure programs; neither asks how the program for a multilingual task is organised.

This paper asks that question. For a frozen model M, task t and output language l, let $K_M(t, l)$ be the length of the shortest prefix that performs t in l at a chosen distortion. How is $K_M(t, l)$ split between telling the model which task to perform and which language to produce? Are the two parts separable, so that a task part and a language part can be assembled? How far apart are the programs for different cells, and do they cluster by task or by language? Where in depth does the model read

Preprint. Under review.

each part? And is the measurement itself trustworthy: does gradient descent find the short programs that the objective prefers?

Our contributions are a measurement design and five results. The design (Section 3) fixes two units, slots and bits, one normalised distortion, two preregistered operating points, and a hierarchical variational prefix whose centre and deviations put task bits and language bits in the same currency. The results (Sections 5–6) are the four findings of the abstract plus a negative one: the information-distance matrix over cells has no cluster structure at one seed. Every number is recomputed from released tables by a notebook that runs in under a minute.

2 Related work

Description length of models and probes Hinton and van Camp [6] introduced the variational (bits-back) code for network weights; Blier and Ollivier [2] compared variational and prequential codes for deep networks and found the prequential one far shorter; Voita and Titov [16] used prequential coding to measure what a probe learns. We use both codes: variational for the rate of a prefix and prequential for the distance between prefixes.

Soft prompts as programs Shaw et al. [14] give the universality construction and the adaptive prior we build on, and report that the objective selects compressible solutions that random-initialisation training fails to find. Cosma [3] measure shortest prompts, discrete and soft, as a complexity. Mu et al. [12] compress instructions into gist tokens; Vu et al. [17] treat learned prompts as task embeddings that predict transfer; Vykopal et al. [18] combine soft language prompts with task adapters. Khashabi et al. [8] show that a working soft prompt need not read as words, which is why every claim here is functional: transfer, composition, conditioning or intervention, never a nearest-token reading.

Task and language inside the model Task vectors [5] and function vectors [15] show that a task can be carried by one hidden vector. Wendler et al. [19] and Dumas et al. [4] locate a language-agnostic concept representation in the middle blocks with language resolved later; our patching probes test where a learned task part and a learned language part are read.

3 Measurement framework

Interpreter, program, distortion The prefix $P \in \mathbb{R}^{k \times d}$ occupies the first k positions of the input embedding sequence; the model's weights, tokenizer and format are fixed. Distortion is the teacher-forced code length L of the target in bits per target token. To compare across cells we normalise per cell,

$$\hat{D} = \frac{L - L_{\text{floor}}}{L_{\text{none}} - L_{\text{floor}}}, \tag{1}$$

where L_{none} is the no-prefix loss and L_{floor} the better of a per-cell LoRA fine-tune and the best prefix run. The operating points $\hat{D} \le 0.25$ and $\hat{D} \le 0.10$ were preregistered from a development pilot; $\hat{D} \le 0.05$ is reported as exploratory. All claims are relative to one model: orderings, differences and ratios within it.

Two units *Slots* are the number k of prefix positions; the slot complexity k^* is the smallest k reaching an operating point, found by sweeping k with free, unpenalised vectors. *Bits* are the description length of the prefix under a prior: each slot is a diagonal Gaussian posterior $q = \mathcal{N}(\mu, \operatorname{diag}\sigma^2)$, trained with its own noise, and its rate is $\mathrm{KL}(q \,\|\, \pi_0)/\ln 2$ with π_0 a diagonal Gaussian fitted to the model's token-embedding matrix (bits-back). The rate–distortion frontier of a cell is the smallest rate among runs that reach an operating point, over k and the KL weight β. Distortion of a variational prefix is scored on posterior samples, because the sample is the coded object (Section 6 shows what goes wrong when the posterior mean is scored instead).

Hierarchy To put task bits and language bits in one currency, a group of cells (the languages of one task, or the tasks of one language) is coded as a centre $c \sim q_c$ with rate $\mathrm{KL}(q_c \,\|\, \pi_0)$ plus one deviation $\delta_m \sim q_m$ per member with rate $\mathrm{KL}(q_m \,\|\, \mathcal{N}(0, \tau^2\sigma_0^2 I))$, $\tau = 1$ in units of the prior scale, so that a deviation costs the same per unit of displacement as a whole prefix. The member's prefix is

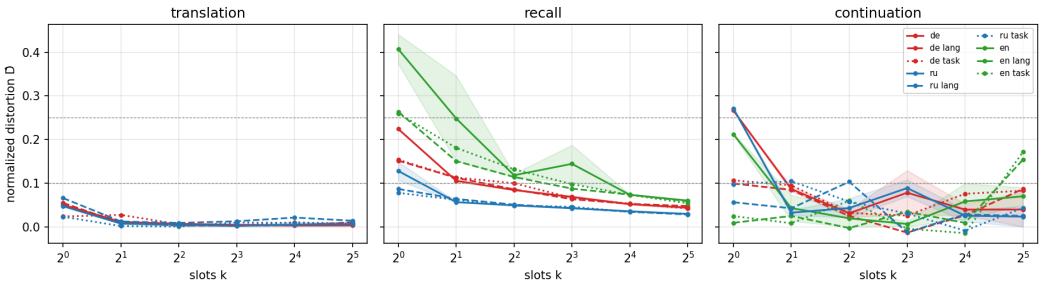


Figure 1: Normalised distortion against the number of slots, Design A, seed means with the seed range shaded. Dashed lines mark the operating points; line style is the conditioning format. Continuation (right) is noise: its floor lies within 0.10–0.16 bits per token of the no-prefix loss.

$c + \delta_m$; the loss is the mean member NLL plus β times the centre rate amortised over all members' tokens and the deviation rate over the member's own.

Derived measurements *Composition*: one tied task part per task and one tied language part per language, concatenated, trained on all cells or with cells held out and assembled zero-shot. *Distance*: $K_M(a \mid b)$ as the prequential cost of training from cell b's prefix toward cell a over a fixed budget, symmetrised and divided by the from-scratch costs [1, 10]. *Mechanism*: the hidden states at the prefix positions of a trained prefix patched into the attention input of one block of a run with a random prefix, for every block; the recovered loss relative to the full effect is the fraction of the program read at that block. *Regime 2*: per-example vectors replacing the input, with and without the task prefix.

4 Experimental setup

Model, data, grid Qwen2.5-1.5B base [13], frozen, bfloat16 on an A100. Translation and continuation use FLORES+ (dev split for training and selection, devtest for reporting); recall uses mLAMA [7] with an English cloze prefix and the object rendered in the target language, split by item. Translation into English is excluded, so the grid has eight cells: `translation-{ru,de}`, `recall-{ru,de,en}`, `continuation-{ru,de,en}`.

Runs Design A: $k \in \{1, 2, 4, 8, 16, 32\}$, two seeds, 1500 steps, batch 16, AdamW with cosine decay; repeated with the target language given, the task given, and the natural-language prompt with no prefix; per-cell LoRA floors. Flat variational: $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$. Hierarchical: both nestings, $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$. Composition: $1{+}1$, $2{+}2$, $4{+}4$ slots with five hold-out sets. Distance: every cell's $k = 4$ prefix toward every cell, 512-example budget in doubling blocks. Regime 2: $m \in \{1, 2, 4, 8\}$ on 100 examples in four cells. Mechanism: eight cells, five probes, 28 blocks. In total 1,805 result rows and about 32 A100 hours.

Two variational protocols The preregistered protocol (*flat*) used learning rate 0.05, the checkpoint with the lowest development distortion at the posterior mean, and σ initialised at five times the prior. Its training histories (Section 6) showed that this overshoots the rate and selects on the wrong statistic. The corrected protocol (*flat2*) uses learning rate 0.01, scores development distortion on samples, selects the checkpoint with the lowest objective (distortion plus β times rate), and initialises σ at the prior. It was run on the five measurable cells at $k \in \{2, 4\}$, $\beta \in \{1, 3\}$ and is the headline where it exists. The hierarchical runs use the corrected initialisation but the preregistered selection rule, a limitation discussed in Section 5.3.

5 Results

5.1 Slots are not a description length

Figure 1 shows $\hat{D}$ against k. Translation reaches $\hat{D} \leq 0.10$ with one slot in every language and conditioning and is flat from there. Recall keeps improving to $k = 32$ on Russian and German

cell	$k^*(0.25)$	$k^*(0.10)$	$k^*(0.05)$, expl.
translation-ru	1	1	1
translation-de	1	1	2
recall-ru	1	2	4
recall-de	1	4	32
recall-en	2	16	not reached

Table 1: Slot complexity: the smallest k whose seed-mean $\hat{D}$ reaches the level.

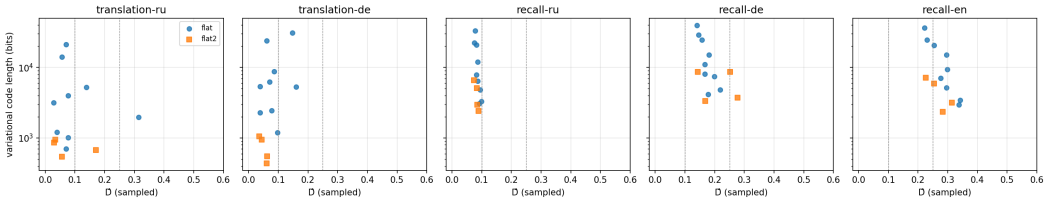


Figure 2: Rate against distortion for every variational run, per cell. Circles: preregistered protocol; squares: corrected protocol. Distortion is scored on posterior samples.

with the budget binding: a 3000-step run on `recall-ru` at $k = 8$ moves $\hat{D}$ from 0.043 to 0.032 and is still falling. Continuation is excluded from every headline claim because its floor lies within 0.10–0.16 bits per token of the no-prefix loss, below the seed variation. Seed gaps are below 0.011 on translation, 0.042 on `recall-ru` and `recall-de`, and 0.20 on `recall-en`. Table 1 gives k^*.

Two observations make the slot count unusable as a description length. First, the natural-language prompt, scored as a context with no prefix, is far worse than one trained slot: on recall it leaves $\hat{D}$ at 0.91–0.96 where one slot leaves 0.13–0.41; on translation 0.12–0.20 against 0.05. One vector of 1,536 numbers carries more than an instruction, so a token is not a unit of information. Second, conditioning saves little on the slot axis: giving the language or the task changes $k^*(0.10)$ in none of the translation cells and in two recall cells (`recall-ru` 2 to 1, `recall-en` 16 to 8, for either conditioning). The savings Δk_{lang} and Δk_{task} average 1.8 slots each over the measurable cells, with 8 of those 9 slots from `recall-en`. The slot axis cannot distinguish telling the model the language from telling it the task; the bit axis can.

5.2 Description length in bits

Table 2 is the headline. A translation program costs about 500 bits. A Russian recall program costs five times that; German and English recall do not reach 10% excess loss within the budget and cost 3,300 and 7,200 bits at 25%. Bootstrap intervals over test examples are narrow: the gain over the no-prefix model of the $\hat{D} \leq 0.10$ frontier runs, evaluated at the posterior mean (per-example bits are stored for the mean only), is 0.52 ± 0.01 bits per token on `translation-ru`, 0.50 ± 0.03 on `translation-de` and 5.69 ± 0.07 on `recall-ru`.

The ordering across tasks within a language follows the learning gap $L_{\text{none}} - L_{\text{floor}}$ (recall 3.5–6.2 bits per token, translation 0.6–0.9), as hypothesised. The ordering across languages does not follow exposure: English recall is the dearest program, not the cheapest, in slots and in bits, and the result survives restricting the three recall test sets to the 4,212 items they share ($\hat{D}$ at $k = 4$: ru 0.049, de 0.079, en 0.122). The English LoRA floor is also the worst of the three (2.58 bits per token against 1.10 for Russian), so what is dear in English is not the language but the facts mLAMA asks about in English.

5.3 Task is the shared factor

Figure 3 and Table 3 compare the codes at $\hat{D} \leq 0.25$, the only level every cell reaches. Grouping by task costs 37% less than grouping by language, and almost all of the difference is recall. Under the same protocol the task-centred code is also 11% cheaper than eight independent programs, but under the corrected protocol the independent programs cost 17,600 bits, less than either hierarchy, and the

cell	corrected (flat2) $\hat{D} \le 0.10$	$\hat{D} \le 0.25$	best $\hat{D}$	prereg. $\hat{D} \le 0.25$
translation-de	436	436	0.036	1,194
translation-ru	542	542	0.030	699
recall-ru	2,419	2,419	0.075	3,101
recall-de	—	3,343	0.143	4,168
recall-en	—	7,155	0.226	24,686

Table 2: Headline frontier: the smallest code length in bits among runs reaching each operating point, under the corrected protocol, with the best distortion any corrected run reaches and the preregistered protocol's value at 0.25.

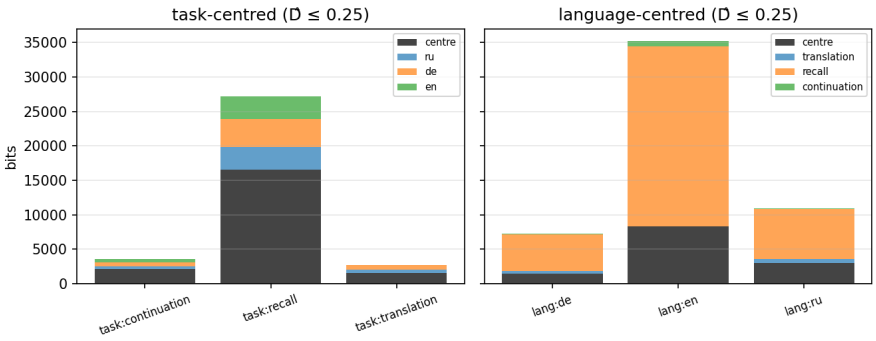


Figure 3: Code length at $\hat{D} \le 0.25$ under the two nestings: the centre is paid once per group, each deviation once per member; the cheapest run whose members all reach the level.

hierarchy has not been rerun with it. Whether sharing saves bits at all is therefore not established by these runs; what Table 3 establishes is the ordering of the two nestings.

The per-member costs explain that ordering. Given the translation centre, a target language costs 485 (ru) or 662 (de) bits: 0.55–0.69 of a standalone program under the same protocol and 0.9–1.5 under the corrected one. Given the recall centre, a language costs 3,300–4,000 bits. In the reverse nesting, given a language centre, continuation costs 110–180 bits (ru, de) or 740 (en), translation 440–630, and recall 5,300 (de), 7,200 (ru) or 26,200 (en). Recall is the program that cannot be shared with anything, and the language-centred code pays for it three times. For translation alone the hierarchy does not pay under either protocol: centre plus two deviations costs 2,733 bits at $\hat{D} \le 0.10$ against 1,893 (preregistered) or 978 (corrected) for two independent programs. In bits, then, the task given the language costs 1.3 (de), 2.2 (ru) and 7.8 (en) times the language given the task on recall, and the two are equal on translation.

5.4 Composition

Six tied parts trained on every cell match Design A at the same total slots on translation ($\hat{D}$ 0.017–0.027 against 0.005–0.007 at 2+2) and cost about twice its distortion on recall (0.08–0.25 against 0.05–0.12). Assembled zero-shot for a held-out cell (Figure 4), the parts close 60–73% of the gap between no prefix and the floor on translation (three hold-outs over two cells, $\hat{D}$ 0.27–0.40) and 14–32% on recall (six hold-outs over three cells, $\hat{D}$ 0.68–0.86). Task and language parts compose partially for translation and barely for recall.

5.5 Distance between programs

Warm-starting from a prefix of the same task costs 0.85 of training from scratch; from the same language or an unrelated cell, 0.95 (Figure 5, left). The silhouette of the task partition on the symmetrised distance is 0.004 and that of the language partition -0.09: the direction is the hypothesised one, the structure is absent, and with seven pairs per group at one seed the difference is not significant. The zero-shot block (right) is the strong signal: a prefix applied to any other cell is roughly as bad as, or

code for all eight cells, $\hat{D} \leq 0.25$	protocol	bits
centre per task + language deviations	prereg. selection	33,500
eight independent programs	prereg.	37,600
centre per language + task deviations	prereg. selection	53,500
eight independent programs	corrected	17,600

Table 3: Three codes for the same eight cells. The hierarchy ran under the preregistered selection rule only; the corrected row uses flat2 on the five measurable cells and the preregistered runs on continuation.

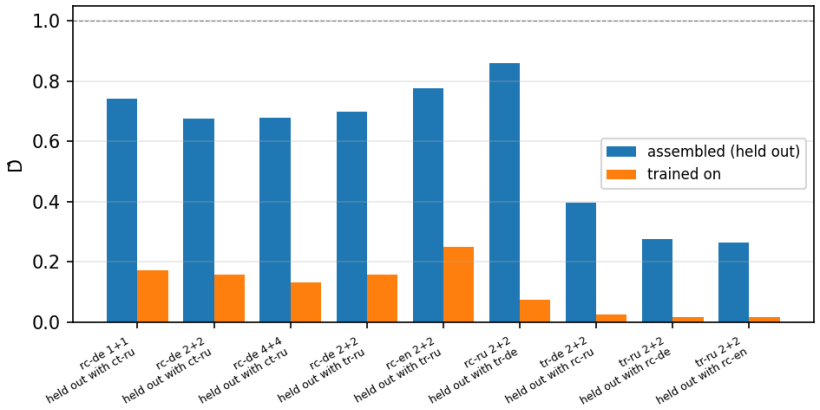


Figure 4: Assembled (held-out) against trained-on $\hat{D}$ of tied task and language parts, per hold-out.

worse than, no prefix ($\hat{D}$ 0.7–1.4, median 0.97, up to 2.5), and a recall prefix applied to continuation is catastrophic ($\hat{D}$ 5–25). The recall program overrides the model's default behaviour rather than adding to it.

5.6 Where the program is read

Patching all blocks reproduces the donor exactly. Patching block 0 alone recovers 70–87% of the effect for translation and 74–94% for continuation, but only 36–58% for recall, which has a secondary peak at block 17 and a smaller one at blocks 10–11 (Figure 6, left). Single-block fractions summed over blocks come to 2.6 for translation and 3.0 for continuation, so their reading is redundant, but 1.3 for recall, which is close to additive. A translation program is copied into the token stream in the first block and not needed afterwards; a recall program is read in several places, deep in the network.

The block of maximal recovery is block 0 for every probe on every cell, so the peak does not separate the parts of a hierarchical prefix. The profiles do. The task centre is read early: on recall its mass over blocks 0–3 is 1.0 and over blocks 14–27 it is 0.5. The language deviation on top of it is read early and again deep: on recall 1.6 early and 1.9 late, with block 17 alone recovering 0.94 of the deviation's effect; on `translation-de` 1.2 and 2.1, with blocks 16 and 22 recovering 0.99 and 0.85. The task deviation under a language centre behaves like the task centre (1.0 and 0.5). There is a depth split by prefix part, in the opposite direction to the language-early, task-middle expectation drawn from Dumas et al. [4]: the task part is read early, the language part early and late. One detail: the recall centre without its English deviation is worse than a random prefix on English, so the recall centre is a non-English program.

5.7 Input compressibility (exploratory)

With the $k = 4$ task prefix, one per-example vector replaces the input to within 10% of the real-input loss on every cell but `translation-ru`, which needs two; without the prefix, translation and continuation need two. There is no rate term on the vectors, so this is a slot count.

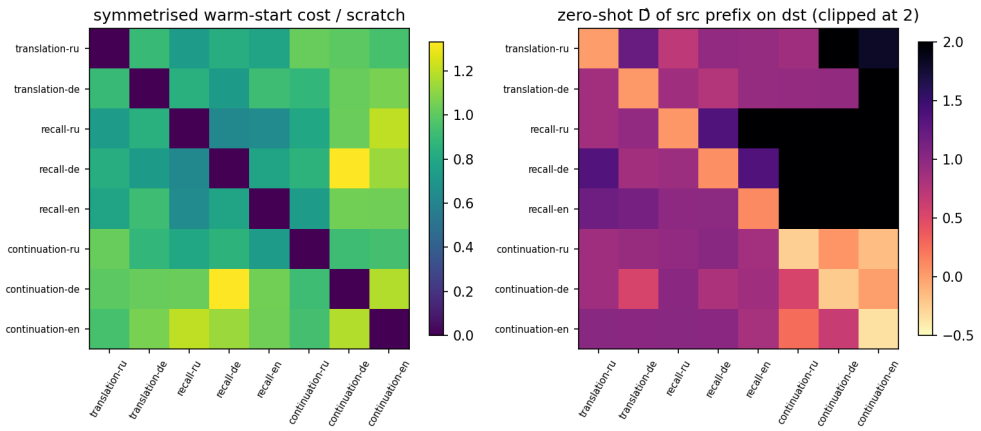


Figure 5: Left: symmetrised warm-start learning cost divided by the from-scratch cost. Right: $\hat{D}$ of a trained prefix applied unchanged to another cell, clipped at 2.

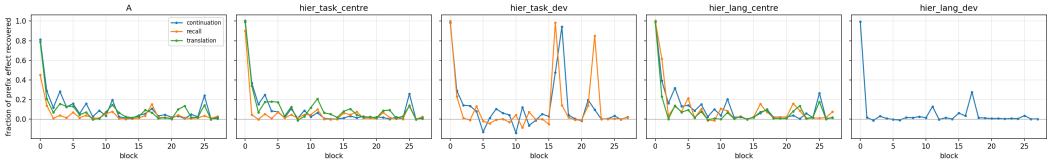


Figure 6: Fraction of a prefix's effect recovered by patching its positions into one block's attention input, mean over cells per task. Probes: `A` (Design A, $k = 4$); `hier_task_centre` and `hier_task_dev` (task centre; language deviation on top of it); `hier_lang_centre` and `hier_lang_dev` (language centre; task deviation on top of it). Fractions are undefined where the donor is not at least 0.1 bits per token better than the recipient.

6 The objective is right, the search is hard

The saved histories of the 27 task-grouped hierarchical runs show three things (Figure 7). With learning rate 0.05, two prior standard deviations per Adam step, the centre KL overshoots early, peaking between steps 100 and 800 by an amount that grows with k: a median peak of 26,000 bits at $k = 2$ (range 6,000–182,000) and 144,000 at $k = 8$ (28,000–549,000), before the cosine schedule lets the KL term pull the means back. In the plotted translation runs the final centre rate is the same at $k = 8$ as at $k = 2$, but the posterior-mean distortion worsens late at $k = 8$, so the distortion-only checkpoint rule selects a step where the rate is ten times the final one; across all runs the selected rate exceeds the final one by a median factor of 1.1–1.2 and up to 20. And the posterior mean is not the coded object: on translation at $\beta = 3$ the mean scores $\hat{D}$ 0.35–0.56 in three of four runs while samples score 0.06–0.17.

The corrected protocol removes all three effects and lowers the measured frontier on every cell, by 22% (`translation-ru`, `recall-ru`), 58–63% (`translation-de`), 20% (`recall-de`) and 71% (`recall-en`), with four runs per cell instead of nine, and the rates become monotone in β. Shaw et al. [14] report that on a parity task the same objective reaches 870 bits from a hand-compiled initialisation but 2.8×10^6 bits from random initialisation, and leave the optimiser as the open problem. At this scale the gap closes mostly by selecting on the objective and stepping at the scale of the prior.

7 Limitations

Two seeds were preregistered; the distance matrix is one seed. The recall budget binds at 1500 steps, so recall rates are upper bounds. Continuation is unmeasurable because the frozen model already performs it. The variational protocol was corrected after the preregistered runs, and the hierarchical

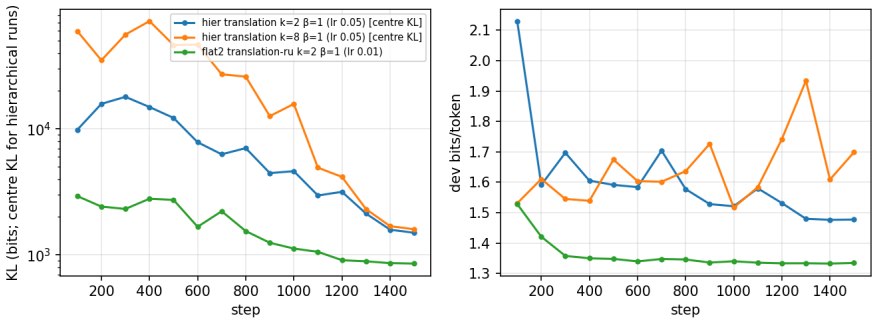


Figure 7: Centre KL (hierarchical runs) or total KL (flat2) and development loss against step for two preregistered-protocol runs and one corrected-protocol run.

runs keep the preregistered selection rule, so their rates carry the selection bias of Section 6 and the hierarchy-versus-independent comparison is open until the hierarchy is rerun under the corrected protocol. The mechanism probes patch attention inputs at the prefix positions; a residual-stream patch would answer a slightly different question. The exposure proxy for the language ordering was not available, so that comparison rests on orderings over three languages.

8 Conclusion

Counting tokens misreads a prompt's complexity; counting bits under a bits-back code does not, and at that currency a frozen 1.5B model needs about 500 bits to be told to translate and 2,400 or more to be told to recall facts. The organisation of those bits is by task: recall programs are language-specific, do not compose and are read deep, translation programs are read in the first block and share a target language for a few hundred bits. The measurement is only as good as the optimiser, and the largest single effect in this study is the one obtained by selecting checkpoints on the objective rather than on distortion. The research version runs the same code over the scale axis and twenty languages, charges $\log_2 |k\text{-grid}|$ bits for the choice of k, adopts the per-column mixture prior of Shaw et al. [14], and reruns the hierarchy under the corrected protocol with three seeds.

Reproducibility Code, result tables, per-example bits, training histories and prefix checkpoints are in the repository (branch `prefixlab`); `notebooks/01_figures.ipynb` regenerates every figure and table from the tables and `prereg.json`.

References

[1] Charles H. Bennett, Péter Gács, Ming Li, Paul M. B. Vitányi, and Wojciech H. Zurek. Information Distance. *IEEE Transactions on Information Theory*, 44(4), 1998.

[2] Léonard Blier and Yann Ollivier. The Description Length of Deep Learning Models. In *Advances in Neural Information Processing Systems*, 2018.

[3] Adrian Cosma. Prompting Complexity: Shortest Prompts for Texts and Behaviors in LLMs. *arXiv preprint arXiv:2607.06145*, 2026.

[4] Clément Dumas, Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. Separating Tongue from Thought: Activation Patching Reveals Language-Agnostic Concept Representations in Transformers. In *Proceedings of ACL*, 2025.

[5] Roee Hendel, Mor Geva, and Amir Globerson. In-Context Learning Creates Task Vectors. In *Findings of EMNLP*, 2023.

[6] Geoffrey E. Hinton and Drew van Camp. Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights. In *Proceedings of COLT*, 1993.

[7] Nora Kassner, Philipp Dufter, and Hinrich Schütze. Multilingual LAMA: Investigating Knowledge in Multilingual Pretrained Language Models. In *Proceedings of EACL*, 2021.

[8] Daniel Khashabi, Xinxi Lyu, Sewon Min, Lianhui Qin, Kyle Richardson, Sean Welleck, Hannaneh Hajishirzi, Tushar Khot, Ashish Sabharwal, Sameer Singh, and Yejin Choi. Prompt Waywardness: The Curious Case of Discretized Interpretation of Continuous Prompts. In *Proceedings of NAACL*, 2022.

[9] Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of EMNLP*, 2021.

[10] Ming Li, Xin Chen, Xin Li, Bin Ma, and Paul M. B. Vitányi. The Similarity Metric. *IEEE Transactions on Information Theory*, 50(12), 2004.

[11] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of ACL*, 2021.

[12] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems*, 2023.

[13] Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.

[14] Peter Shaw, James Cohan, Jacob Eisenstein, and Kristina Toutanova. Bridging Kolmogorov Complexity and Deep Learning: Asymptotically Optimal Description Length Objectives for Transformers. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.22445.

[15] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2024.

[16] Elena Voita and Ivan Titov. Information-Theoretic Probing with Minimum Description Length. In *Proceedings of EMNLP*, 2020.

[17] Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou, and Daniel Cer. SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. In *Proceedings of ACL*, 2022.

[18] Ivan Vykopal, Simon Ostermann, and Marián Simko. Soft Language Prompts for Language Transfer. In *Proceedings of NAACL*, 2025.

[19] Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. Do Llamas Work in English? On the Latent Language of Multilingual Transformers. In *Proceedings of ACL*, 2024.

A Preregistered comparisons

H1	task ordering supported (recall far above translation); language ordering reversed on recall (English dearest), robust to common items; exposure proxy unavailable
H2	equal on the slot axis (1.8 slots each, 8 of 9 from one cell); in bits, task given language costs 1.3–7.8$\times$ language given task on recall, equal on translation
H3	translation composes partially (60–73% of the gap zero-shot), recall barely (14–32%); hierarchy vs independent programs not established; Chinese not run
H4	silhouette 0.004 vs -0.09: task direction, no structure, not significant
H5	block 0 for both; the language deviation is read again at blocks 16–17 and the task centre is not: a split by part, opposite in direction to the hypothesis
H6	one vector suffices with the task prefix; per-token variant not computed; no rate term

Table 4: Outcome of each preregistered comparison.

B Run inventory

273 Design A, baseline and floor rows (6 values of k, 2 seeds, 3 conditionings, 8 cells); 72 flat and 20 flat2 rows; 144 hierarchical rows from 54 joint runs; 72 composition rows from 10 joint runs; 72 distance rows; 32 regime 2 rows; 1,120 mechanism rows; one 3000-step recall run. Training: 1500 steps, batch 16, AdamW, learning rate 0.05 (0.01 for flat2), 50 warm-up steps, cosine decay, gradient clip 1.0, development evaluation every 100 steps. LoRA floors: rank 16 on the attention projections, learning rate 2×10^{-4}, 1500 steps.